multiprocessing.Manager() → context._default_context.Manager

PID: 556

Manager

```python
def Manager(self):
    '''Returns a manager associated with a running server process

    The managers methods such as `Lock()`, `Condition()` and `Queue()`
    can be used to create shared objects.
    '''
    from .managers import SyncManager
    m = SyncManager(ctx=self.get_context())
    m.start()
    return m
```

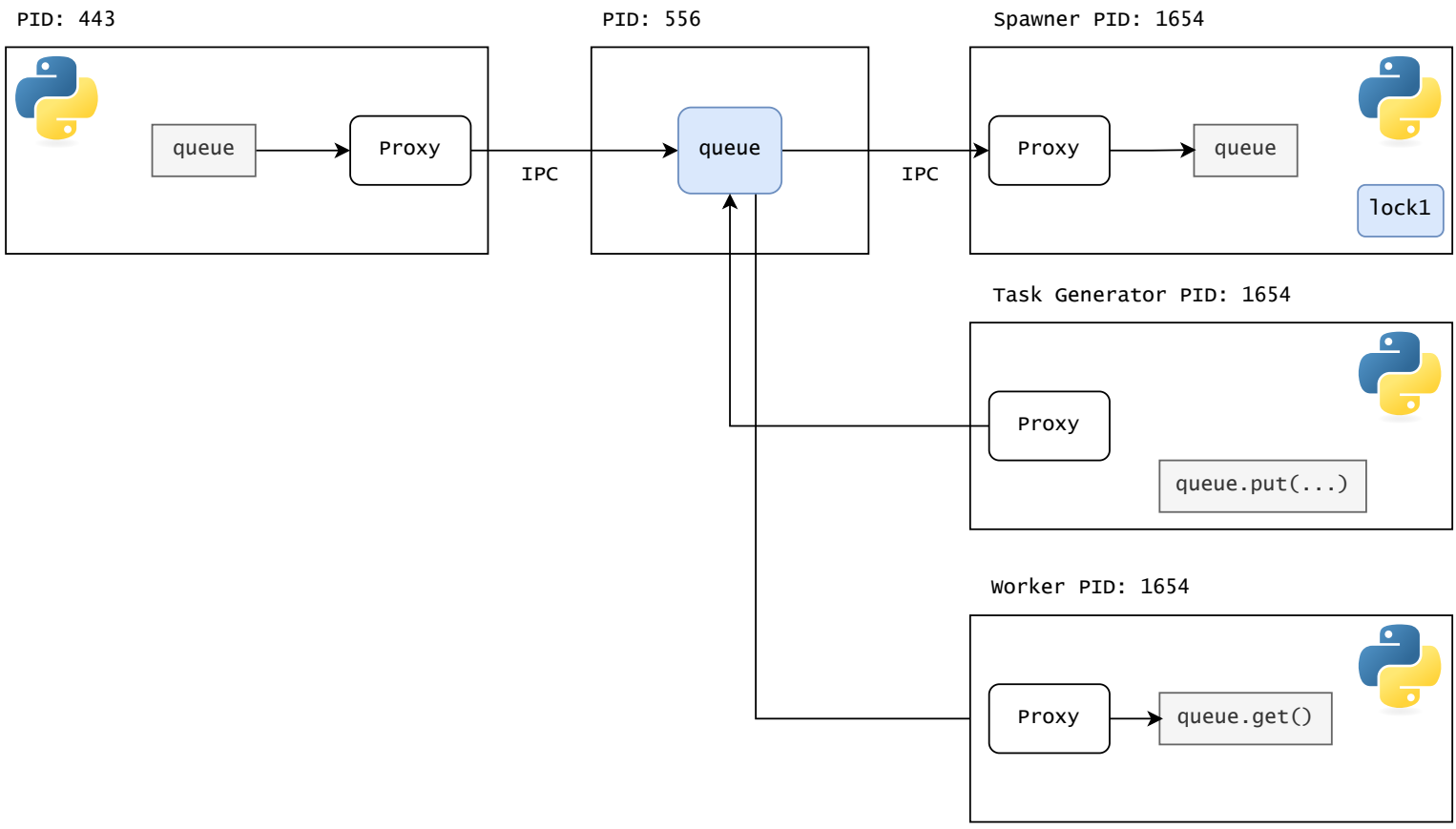
```python
class SyncManager(BaseManager):
    '''
    Subclass of `BaseManager` which supports a number of shared object types.

    The types registered are those intended for the synchronization
    of threads, plus `dict`, `list` and `Namespace`.

    The `multiprocessing.Manager()` function creates started instances of
    this class.
    '''

SyncManager.register( typeid: 'Queue', queue.Queue)
SyncManager.register( typeid: 'JoinableQueue', queue.Queue)
SyncManager.register( typeid: 'Event', threading.Event, EventProxy)
SyncManager.register( typeid: 'Lock', threading.Lock, AcquirerProxy)
SyncManager.register( typeid: 'RLock', threading.RLock, AcquirerProxy)
SyncManager.register( typeid: 'Semaphore', threading.Semaphore, AcquirerProxy)
SyncManager.register( typeid: 'BoundedSemaphore', threading.BoundedSemaphore,
                      AcquirerProxy)
SyncManager.register( typeid: 'Condition', threading.Condition, ConditionProxy)
SyncManager.register( typeid: 'Barrier', threading.Barrier, BarrierProxy)
SyncManager.register( typeid: 'Pool', pool.Pool, PoolProxy)
SyncManager.register( typeid: 'list', list, ListProxy)
SyncManager.register( typeid: 'dict', dict, DictProxy)
SyncManager.register( typeid: 'Value', Value, ValueProxy)
SyncManager.register( typeid: 'Array', Array, ArrayProxy)
SyncManager.register( typeid: 'Namespace', Namespace, NamespaceProxy)

# types returned by methods of PoolProxy
SyncManager.register( typeid: 'Iterator', proxytype=IteratorProxy, create_method=False)
SyncManager.register( typeid: 'AsyncResult', create_method=False)
```

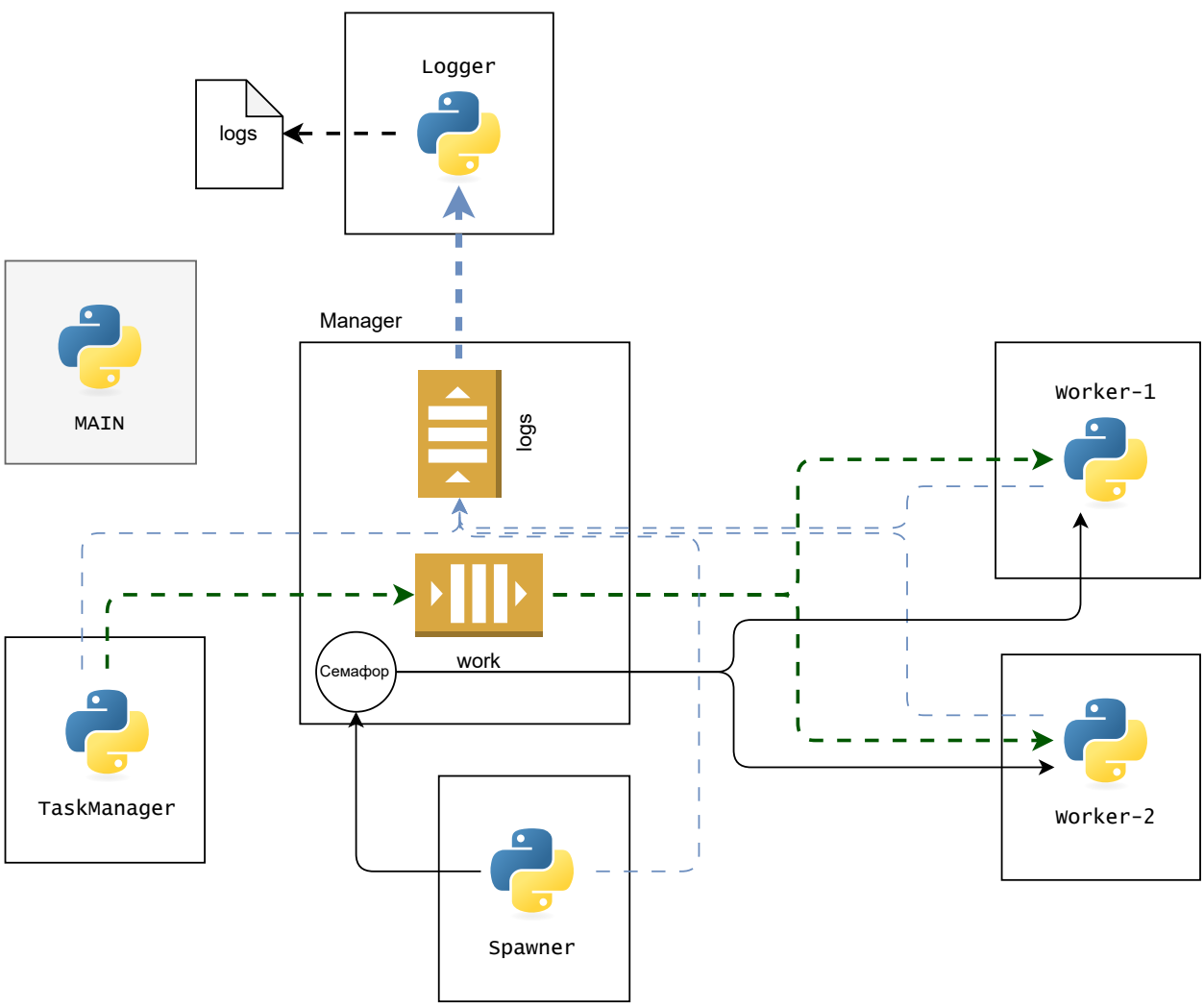
```python
>>> m = multiprocessing.Manager()
>>> m.address
'\\\\.\\pipe\\pyc-7448-0-vce4x1ys'
```

named pipe

```python
>>> m = multiprocessing.Manager()
>>> m.address
'/tmp/pymp-eg0fwhm8/listener-g7wmhm_g'
```

UNIX socket


PID: 443
queue
Proxy
IPC
PID: 556
queue
IPC
Spawner PID: 1654
Proxy
queue
lock1
Task Generator PID: 1654
Proxy
queue.put(...)
Worker PID: 1654
Proxy
queue.get()


Logger
logs
MAIN
Manager
logs
work
Семафор
TaskManager
Spawner
Worker-1
Worker-2